

Domain 1

GigShield

AI Companion for Fair Wages & Worker Safety

Background

Delivery riders, cab drivers, and other gig workers rarely have tools built for them — most apps are built for the platforms, not the people doing the work. Underpayment, unsafe routes, and a total lack of institutional support are everyday realities, and there's almost no accessible tech addressing it.

Problem Statement

Build an AI-powered companion for gig workers that goes beyond simple earnings tracking — one that acts as a financial coach, a safety net, and a rights advisor all in one. It should help a worker understand not just how much they earned, but whether it was fair, safe, and sustainable, and step in with real support the moment something looks wrong.

Core Requirements

- Let the worker log each job (fare, distance, time) manually, or scan a screenshot of their delivery/ride app using OCR to auto-extract the earnings data
- A simple fairness-check model that flags a job as "possible underpayment" by comparing the actual payout against an expected fair-rate benchmark for that distance/time (a small reference dataset is enough)
- An AI chatbot (LLM API) that explains, in simple language, things like "is this fare fair?", "what are my rights?", or "how do I raise a complaint?"
- A dashboard summarizing weekly earnings, flagged underpayments, and total hours worked
- Multi-platform earnings aggregator let the worker log jobs from more than one gig app (e.g., a food delivery app and a ride-hailing app) and see all their earnings unified in a single dashboard
- An AI-generated weekly insight summary that goes beyond raw numbers e.g., "You earned 12% less this week, and most of the underpayment happened during night shifts"

Bonus Features (Extra Points)

- Voice-based interaction (speech-to-text) so low-literacy workers can use it without typing
- A simple AI-generated route safety score based on time of day and area (use any reasonable proxy data)
- Multilingual support using a translation API
- AI-generated complaint draft that the worker can copy-paste or send, based on a flagged underpayment
- A fatigue/burnout detector that flags unusually long consecutive working hours and has the AI gently check in and suggest a break
- An "I feel unsafe" trigger a single tap that has the AI draft and prepare a location-sharing alert message for a trusted contact

- A savings goal tracker where the worker sets a target, and the AI adjusts its suggestions based on real earnings trends
- A community fairness benchmark simulates crowdsourced fare data from multiple workers to build a more accurate fair-rate baseline than a single static reference

Suggested Tech Stack

Any LLM API (OpenAI / Gemini / HuggingFace), OCR API (Google Vision / Tesseract), Python for the fairness-check logic, React or Flutter for the frontend, Firebase/Twilio for alerts and notifications if implementing the safety trigger

Deliverable

Working prototype + demo video showing: job logging/screenshot scan → fairness check → chatbot Q&A → earnings dashboard

Domian 2

SignBridge

AI Companion for Real-Time Communication & Navigation

Background

Accessible technology is often expensive, hardware-dependent, or simply doesn't exist for everyday situations. A hearing-impaired student can't always find someone who knows sign language, and a visually impaired person can't always tell what's around them. Both are solvable today with AI that's freely available, yet almost nobody builds for this space compared to how much attention it deserves.

Problem Statement

Build an AI-powered accessibility companion that bridges the gap between the hearing and non-hearing world in real time translating sign language into natural, grammatically correct sentences, describing a visually impaired user's surroundings from a single photo, and acting as an always-available interpreter for moments when no human one is around.

Core Requirements

- Hand gesture recognition using a pretrained model (e.g., MediaPipe Hands + a lightweight classifier) to translate a basic set of sign language gestures into individual words
- Sign-to-sentence mode: instead of displaying isolated recognized words, chain them together using an LLM to reconstruct a natural, grammatically correct sentence bridging sign language grammar structure to spoken language grammar
- A speech-to-text and text-to-speech pipeline so a hearing person and a non-hearing person can hold a full two-way conversation through the app
- A "describe my surroundings" feature: the user takes a photo, and an AI vision/captioning model describes what's in it in simple spoken language (e.g., "a chair to your left, a doorway ahead")
- A screen-reader-friendly, accessible interface accessibility of the app itself matters as much as its features

Bonus Features (Extra Points)

- Personal sign trainer: let the user teach the app 3-5 custom signs on the spot using a handful of sample recordings (a simple few-shot/fine-tuning approach), so the vocabulary can grow beyond the default set
- Scene navigation assistant: extend the surroundings feature into basic turn-by-turn guidance (e.g., "obstacle approximately 2 steps ahead") using object detection and relative bounding-box size as a rough distance estimate
- Emotion-aware captioning: detect basic tone/emotion in the hearing person's speech (e.g., urgent, calm, happy) and reflect it in the captions shown to the non-hearing user, so emotional context isn't lost in translation

- An emergency distress gesture: recognizing a specific pre-defined hand signal as a request for help, which automatically alerts a trusted contact
- Regional sign language toggle: support switching between ISL (Indian Sign Language) and ASL (American Sign Language) gesture sets, reflecting the diversity of a national-level audience
- Text-to-Braille pattern conversion (visual representation only, no physical hardware required)

Suggested Tech Stack

MediaPipe / OpenCV for gesture recognition, any LLM API (OpenAI / Gemini / HuggingFace) for sentence reconstruction and emotion-aware captioning, HuggingFace vision/captioning models for scene description and object detection, Python, React or Flutter with speech APIs

Deliverable

Working prototype + demo video showing: gesture-to-sentence translation → two-way speech/text conversation loop → photo-based surroundings description, with at least one bonus feature demonstrated live

Domain 3

AgriSense

AI Farm Companion for Crop Health & Fair Pricing

Background

Small farmers rarely have access to an agronomist on demand, and often sell their produce for less than it's worth simply because they don't have reliable price information. Most agri-tech tools assume large-scale farms and internet-heavy usage, not the reality of a smallholder farmer with a basic smartphone and a single field to manage.

Problem Statement

Build an AI-powered farm companion that acts like an agronomist, a market analyst, and a weather advisor all in one diagnosing crop health from a photo, explaining the diagnosis in plain conversational language, and helping the farmer decide not just what's wrong with the crop, but when and where to sell it for the best possible return.

Core Requirements

- Crop disease/pest detection: the farmer uploads a photo of a leaf or plant, and a pretrained image classification model identifies the likely issue with a confidence score.
- An AI chatbot (LLM API) that explains the diagnosis in simple language and suggests practical next steps or remedies.
- A basic market price trend view for a couple of crops, using a simple regression model on simulated historical price data.
- Smart sell/hold advisor: an AI agent that uses function calling / tool use to pull together the price trend, a simulated weather forecast, and the crop's perishability, then reasons through and recommends "sell now" or "hold for a few more days" with a clear explanation.
- Voice input support, since many users will be more comfortable speaking than typing.

Bonus Features (Extra Points)

- Multi-photo field scan let the farmer scan several plants from the same field in one session, and aggregate the results into a single field health score (e.g., "78% healthy, 22% showing early blight symptoms").
- Yield-loss estimation: combine disease severity with estimated affected area to project a rough potential yield loss percentage, giving the farmer a sense of urgency.
- Community outbreak map: simulate scan reports from multiple farmers in a region and use simple clustering/aggregation to flag if a disease appears to be spreading locally, not just on one farm.
- Multilingual/vernacular language support using a translation API.
- A community Q&A section where farmers can ask questions, with AI-suggested answers based on similar past questions.

- A concept mockup for WhatsApp-bot style delivery, since that's how many farmers already communicate digitally.

Suggested Tech Stack

HuggingFace/TensorFlow.js pretrained vision models, any LLM API (OpenAI/Gemini) with function calling support for the sell/hold advisor, a public or mock weather API, Python for the price-trend regression and clustering logic, React or Flutter

Deliverable

Working prototype + demo video showing: crop photo upload → AI diagnosis → chatbot explanation → price trend view → sell/hold recommendation with reasoning, with at least one bonus feature demonstrated live